

Software Atelier 4: Software Engineering Project (Spring 2024)

Instructors: Dr. Andrea Mocci

TAs: Stefano Campanella, Gianmarco De Vita , Masoud Jamshidiyantehrani,

Niccolò Puccinelli, Lamberto Ragnolini

Batch 2 - Part 2, User Stories (v1, 55/70 points)



1. As a student, after an attempt, I can get an explanation on the received feedback on a question, so that I can better understand why the answer was wrong. 5 pts

Use an automatic generative method, based on AI, to generate an explanation on a wrong answer, when given a feedback after an attempt. The answer is stored in the attempt so that when the student reopens the attempt, they will see the original answer.

2. As a student, I can ask the AI a specific question on a resource on the materials, so that I can clarify some doubt that I have. 5 pts

The question should be made on a highlighted piece of text of the resource, and it should work on anything internal (in this case, markdown pages).

3. As a student, I can rate any AI answer I received or that I can see, so that I can help identify the best ones. 2.5 pts

You can choose the form of rating you prefer.

4. As a teacher, I can see the answers given by the AI on materials or on exercise attempts. 2.5 pts

As a teacher, I should be able to see the answers given by the AI on materials and on exercise attempts.

5. As a teacher, I can make the answers given by the AI public, so that other students can see them. 5 pts

As a teacher, I can make some of the AI answers public, so that the students can see them. In the case of attempt clarifications, the answers should be shown only if the student makes the same mistake as the context of a previously given AI explanation. The UI should take into account the ratings. As soon as an answer is public, the other students can rate it as well.

6. As a teacher, I can specify a Exercise With Variants, so that every attempt can be slightly different. 15 pts

An exercise with variants is another form of exercise where each question has variants, which are lists of possible question alternatives. All variants of a questions are of the same type (*e.g.*, boolean question), but their text and the correct answer may be different. When attempting an exercise, the system will select one specific variant of each questions to submit to the students.

For example, an exercise has a common text that asks to consider a piece of code, or a fragment of a UML diagram. The first question could be taken from a list of 5 variants of a boolean question; the second question could be fixed, with a single variant; and the third question could be an open answer question with 3 variants.

7. As a teacher, I can get an automatic grade of an open answer question done by the AI, so that I can check it to confirm it or not.	10 pts
Open answer questions should be now automatically graded by the AI. From the point of view of the student nothing changes (regarding the feedback), but the teacher can inspect the grading and the reasoning done by the AI. The correct answer is not anymore a list of keywords, but a sentence in natural language that the teacher provides upfront. The AI must be used to check that the student' answer is compatible with the teacher's answer, that can confirm the grade or not.	
8. As a teacher, I can comment on an answer given by an AI, so that I can clarify some partially incorrect aspects.	5 pts
As a teacher, I can comment on some answers given by the AI in order to clarify certain aspects which are incorrect or misleading. Commenting is usually done on public answers.	
9. As a teacher, I ask the AI to generate a variant for a question.	5 pts
As a teacher, I can ask the AI to generate a variant for a question, which should be similar but different than the ones existing in the variant list.	

Suggestions for Planning and Implementation

- You should not implement the changes in the frontend after the changes in the backend. First, try to identify the changes in the API provided by the backend, and construct some fake data or stubs that the frontend can work with. Team members should be able to work on the changes in the frontend even when the backend features are not fully implemented. When features are implemented, replace fake data with fetch calls to the backend, and ensure that everything works properly.
- Try to always provide some basic tests with each new feature you implement. You may complete testing in a second step, but avoiding merging into dev a feature that has not been tested at all.
- The user story 6 has a lot of implications on your domain model. This should be prioritized.
- Overall, there are a lot of interactions in these user stories. Implement them incrementally, test them, and try to find possible interactions that are not evident.
- You should use Spring AI as presented during the lab.
- AIs can fail. You should take into account that possibility.
- Testing services which use AI can be tricky. You should mock certain components.
- Create an issue for each user story, both in the backend and in the frontend. For each of these issues try to split it into different tasks. For example, you should include things like:
 - Extending the domain model in the backend;
 - Understand the changes in the controller;
 - Study/sketch the changes to the UI in the frontend;
 - ...